



UNIVERSITÀ
DEGLI STUDI
DI MILANO

Introduzione ai db NoSQL

LA STATALE

Prof. Stefano Montanelli

Yesterday vs. Today Needs

- The needs around data storage and management strongly changed over time
- In the 1960s and 1970s:
 - We had limited and expensive computing/network resources
 - The data volume to manage was limited
 - The rigid structure of relational databases represented a plus for the design of effective solutions of data organization
 - Constraints were perceived as a support to avoid data inconsistencies

Yesterday vs. Today Needs

- Today, in certain application contexts, the rigid data organization of relational databases is perceived as an obstacle rather than a benefit (e.g., web data collection)
- Personal/portable computing devices are widespread with pervasive networking support
- The data volume is unlimited
- The availability of flexible data models represents a plus for effective data organization and storage
- Constraints are more than necessary in many situations (e.g., FK on a tweet insert)

“One size fits all”

- *The last 25 years of commercial DBMS development can be summed up in a single phrase: “One size fits all”*
- *This phrase refers to the fact that the traditional DBMS architecture (originally designed and optimized for business data processing) has been used to support many data-centric applications with widely varying characteristics and requirements*
- *We argue that this concept is no longer applicable to the database market, and that the commercial world will fracture into a collection of independent database engines, some of which may be unified by a common front-end parser*
- Stonebraker, M. and Cetintemel, U., One Size Fits All: an Idea whose Time has Come and Gone, in Proc. of ICDE, 2005

Origin of NoSQL databases

- The term NoSQL appears the first time in 1998 by Carlo Strozzi to name his OpenSource, LightWeight database without any SQL interface
- According to a widely-accepted definition, NoSQL means **Not Only SQL**
- In the early 2009, Eric Evans, a Rackspace employee, reused this term to refer DBs that are:
 - non-relational
 - distributed
 - not conforming to ACID properties

Origin of NoSQL databases

Alternative definition:

- *“NoSQL are next generation databases mostly addressing some of the points: being non-relational, distributed, open source and horizontally scalable”* <http://nosql-database.org>

Main features of NoSQL DBs

Horizontal scalability

- NoSQL DBs are designed to scale horizontally, which means that the system can be expanded while it is operational, by adding more nodes for storage and processing as data volume increases

Availability, Replication

- NoSQL DBs are designed to enforce availability through data replication in a transparent way: i) improved read performance; ii) complex write performance (since an update must be applied to all the copies)

Eventual consistency

- NoSQL DBs usually improve write performance through the use of a more “relaxed” form of consistency called eventual consistency (a.k.a. optimistic replication)

Eventual consistency - example

- User A watches the weather report and learns that it's going to rain (the report is a reliable source of information)
- User A tells User B that the weather is going to rain
- User C tells User D that the weather is going to be sunny
- User B tells User C that the weather is going to rain
- User C tells User D that the weather is going to rain

Eventually, all the people know the truth, but in the meantime User D came away with an outdated information

Main features of NoSQL DBs

Horizontal fragmentation

- NoSQL data files can have many millions of records since file records are horizontally partitioned (*data sharding*) to distribute the data access load
- Suitable combination of data sharding and shard replication improve load balancing and system availability

Schema is not required

- NoSQL DBs allow self-describing, semistructured (e.g., JSON) data. There could be a partial schema, BUT the schema is NOT required
- Data constraints must be programmed at the application level, into the program code

Classification of NoSQL databases

We can distinguish NoSQL databases in the following main categories:

- *Key-Value stores* (e.g., DynamoDB, Cassandra, Berkeley DB Redis)
- *Document-based systems* (e.g., MongoDB, CouchDB)
- *Column-family systems* (e.g., Google BigTable, Amazon's SimpleDB)
- *Graph-oriented systems* (e.g., Neo4j, Sones, InfinityGraph)

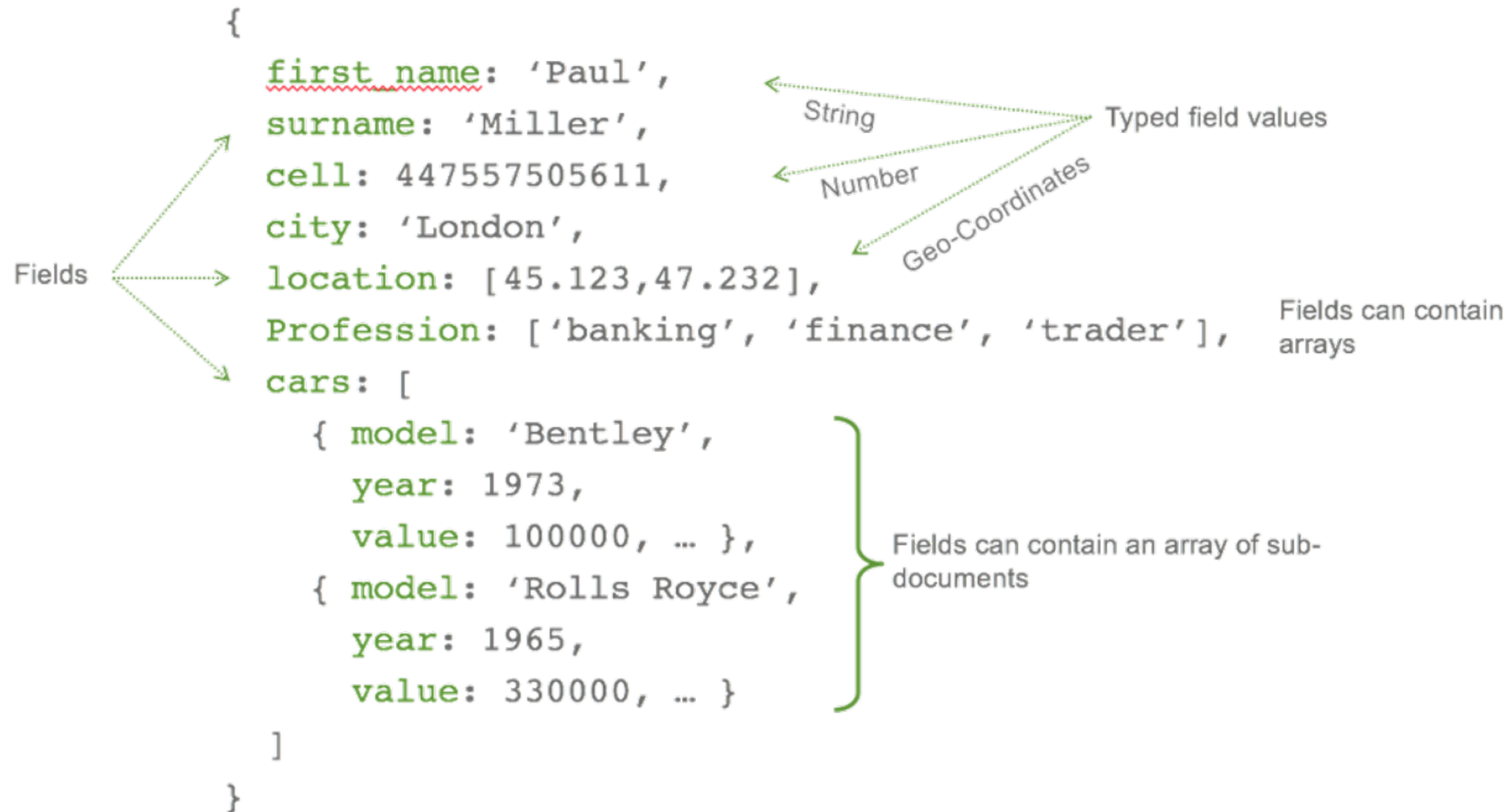
Document-based systems

- Document-based systems are based on the notion of collection and document
- A **document** is a data item with a flexible notion of structure
- A document is self-describing both schema and data
- Documents have a hierarchical structure organization of inter-related data elements
- Documents can be specified in various formats (e.g., JSON - JavaScript Object Notation documents)

Document-based systems

- Document-based systems are based on the notion of collection and document
- A **collection** is a schema-free group of similar documents
- A collection can group documents with similar, but different structure
- Two documents in a collection can have different data elements

Document-based systems





UNIVERSITÀ
DEGLI STUDI
DI MILANO

Il sistema MongoDB

LA STATALE

Prof. Stefano Montanelli

Essentials

- MongoDB (name derived from «humongous») is a cross-platform, document-oriented database engine
- MongoDB has been firstly released in 2007 and it progressively gained popularity in the framework of database solutions
- <http://db-engines.com>

Essentials

- MongoDB is made up of databases which contain **collections**
- A collection shares enough in common with the notion of table in a relational database
- A collection is made up of **documents**
- A document shares enough in common with the notion of tuple/record in a relational database

Essentials

- Each document is made up of **fields**
- A field shares enough in common with the notion of attribute in a relational database
- Collections can be **indexed**, which improves lookup and sorting performance
- Indexes in MongoDB function mostly like their RDBMS counterparts

Serialization format

- MongoDB employs **BSON** (Binary JSON – Binary Javascript Object Notation)
- BSON is a binary serialization format used i) to store documents, and ii) to enforce remote procedure calls
- The MongoDB syntax is case sensitive

Identifiers

- Each document has a predefined **_id** field which represents the primary key of the document within the collection
- The value of the **_id** field is unique inside the collection
- If not inserted, **_id** is automatically generated to provide a unique ObjectId

References

- References across different documents (also when belonging to different databases) can be represented in two ways
- **Normalized way.** The field value of a document contains the value of the `_id` field of the referenced object (similar to the notion of foreign key in relational databases)

Example of normalized reference

```
db.hospitals
{
  _id: ObjectId('F1X'),
  description: 'General Hospital',
  address: 'Via Sforza 35, Milano'
},
{
  _id: ObjectId('F2X'),
  description: 'Sacco Hospital',
  address: 'Via Grassi 74, Milano'
}
```

```
db.doctors
{
  _id: ObjectId('Ak31'),
  name: 'Jordan',
  surname: 'Belfort',
  specialization: 'Ophthalmology'
},
{
  _id: ObjectId('AN77'),
  name: 'Donnie',
  surname: 'Azoff',
  specialization: 'Urology'
}
```

```
db.hospitalizations
{
  _id: ObjectId('2349384533'),
  name: 'Homer Simpson',
  hospitalizations: [{
    hospital: ObjectId('F2X'),
    doctor: ObjectId('Ak31'),
    h_date: new Date(2019,10,12),
    disease: 'strabismus'
  },
  {
    hospital: ObjectId('F2X'),
    doctor: ObjectId('AN77'),
    h_date: new Date(2018,9,9),
    disease: 'gallstone'
  }
  ]
});
```

References

- References across different documents (also when belonging to different databases) can be represented in two ways
- **Denormalized way.** The field value of a document contains the entire referenced object (data embedding)

db.hospitalizations

```
{
  _id: ObjectId('2349384533'),
  name: 'Homer Simpson',
  hospitalizations: [{
    hospital: {
      code: 'F2X',
      description: 'General Hospital',
      address: 'Via Sforza 35, Milano'
    },
    doctor: {
      id: 'Ak31',
      name: 'Jordan',
      surname: 'Belfort',
      specialization: 'Ophthalmology'
    },
    h_date: new Date(2019,10,12),
    disease: 'strabismus'
  }],
  {
    hospital: {
      code: 'F2X',
      description: 'General Hospital',
      address: 'Via Sforza 35, Milano'
    },
    doctor: {
      id: 'AN77',
      name: 'Donnie',
      surname: 'Azoff',
      specialization: 'Urology'
    },
    h_date: new Date(2018,9,9),
    disease: 'gallstone'
  }
}
});
```



The aggregation pipeline

- «*The aggregation pipeline is a framework for data aggregation modeled on the concept of data processing pipelines. Documents enter a multi-stage pipeline that transforms the documents into aggregated results*»
- <https://docs.mongodb.com/manual/core/aggregation-pipeline/>

The aggregation pipeline

- The MongoDB aggregation pipeline consists of stages
- Each stage transforms the documents as they pass through the pipeline
- Pipeline stages do not need to produce one output document for every input document (e.g., some stages may generate new documents or filter out documents)
- The stages in the pipeline are executed in sequence as they appear
- The output of a stage is the input of the subsequent one